

Intelligent Document & Invoice Processing System

Project Architecture, Implementation Plan, Technology Stack and Development Roadmap

Project type: End-to-end Computer Vision + Document AI + Full-Stack Application

Primary goal: Convert invoices and business documents into reliable, structured digital data while preserving the original document and providing visual verification.

1. Project Overview

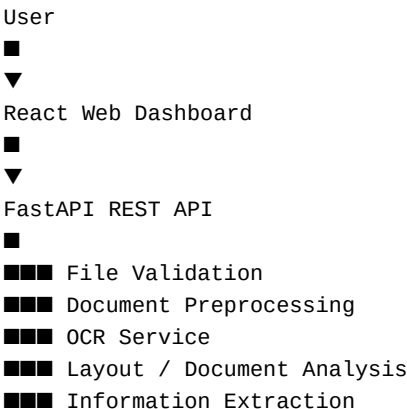
The system will accept invoice and document files such as images and PDFs, preprocess them, extract text and layout information using OCR, identify important fields, convert the results into structured JSON, store the results in PostgreSQL, and present them through a React web dashboard. The backend will be implemented with FastAPI and the application will be containerized with Docker.

2. Core Objectives

- Accept common document formats such as JPG, PNG and PDF.
- Improve document quality through preprocessing, resizing, denoising, thresholding and deskewing.
- Extract text and bounding-box coordinates using OCR.
- Identify document type and important invoice fields.
- Extract structured information such as invoice number, date, vendor, customer, line items, tax and total.
- Show extracted information alongside the original document.
- Visually highlight the source region used for each extracted field.
- Allow users to correct extracted values before final storage.
- Store documents, processing status and extracted data in PostgreSQL.
- Expose processing functionality through a documented REST API.
- Package the complete application using Docker and Docker Compose.

3. High-Level Architecture

The architecture follows a modular pipeline so that OCR, document analysis, extraction, storage and presentation can be developed and tested independently.



■■■ Result Validation
 ■
 ■■■■■■■■■■■■■■■■■■■■■■ PostgreSQL
 ■
 ■■■■■■■■■■■■■■■■■■■■■■ File / Document Storage

4. Technology Stack

Layer	Technology	Purpose
Frontend	React + JavaScript	Dashboard, upload, results, visual verification
Styling	CSS	Responsive and clean user interface
Backend	Python + FastAPI	REST APIs, orchestration and validation
Computer Vision	OpenCV	Preprocessing and image operations
OCR	PaddleOCR	Text recognition and bounding boxes
Document AI	Transformers / LayoutLMv3 or Donut	Advanced document understanding
ML Framework	PyTorch	Running and integrating AI models
Database	PostgreSQL	Persistent metadata and extracted results
Containerization	Docker + Docker Compose	Reproducible development and deployment
Version Control	Git + GitHub	Source control and portfolio

5. Processing Pipeline

Step 1 — Upload

The React application sends an image or PDF to FastAPI using multipart/form-data.

Step 2 — Validation

The backend verifies file type, size, extension and basic document integrity.

Step 3 — Preprocessing

OpenCV improves the input through grayscale conversion, denoising, thresholding, resizing and deskewing where necessary.

Step 4 — OCR

PaddleOCR detects text and returns recognized text, confidence and coordinates.

Step 5 — Layout Analysis

Text coordinates and document regions are used to understand headers, tables, labels and values.

Step 6 — Information Extraction

Rules are used first for predictable fields. Transformer-based document models can later handle more complex layouts.

Step 7 — Validation

Extracted values are checked for expected formats and arithmetic consistency, such as $\text{subtotal} + \text{tax} \approx \text{total}$.

Step 8 — Storage

The document metadata and structured results are stored in PostgreSQL.

Step 9 — Visualization

The React dashboard displays fields, confidence values and highlighted source regions.

Step 10 — Human Verification

The user can correct uncertain fields before marking the document as verified.

6. Functional Modules

Document Upload Module — Upload, validation, file metadata and processing status.

Preprocessing Module — Image cleanup and normalization before OCR.

OCR Module — Text recognition and coordinate extraction.

Document Classification Module — Identify invoice, receipt or other supported document types.

Field Extraction Module — Extract invoice number, date, vendor, customer, totals and line items.

Table Extraction Module — Convert detected table rows into structured item records.

Verification Module — Display confidence and source regions and allow manual correction.

Storage Module — Persist documents, metadata, OCR results and structured fields.

API Module — Provide REST endpoints for upload, processing, retrieval and deletion.

Dashboard Module — Present documents, results, status and analytics.

7. Suggested Project Structure

```
document-ai/  
  frontend/  
    src/  
      components/  
      pages/  
      services/  
      App.jsx  
      package.json  
  backend/  
    app/  
      api/  
      core/  
      models/  
      schemas/  
      services/  
      preprocessing/  
      ocr/  
      extraction/  
      validation/  
      main.py  
      tests/  
      requirements.txt  
  models/  
  uploads/  
  docker/  
  docker-compose.yml  
  .env.example  
  README.md
```

8. Database Design

Table	Important fields
documents	id, filename, type, status, created_at, processed_at
invoices	id, document_id, invoice_number, invoice_date, vendor, customer, subtotal, tax, total
invoice_items	id, invoice_id, description, quantity, unit_price, line_total
ocr_results	id, document_id, text, confidence, bounding_box
verification	id, document_id, verified, corrected_by, verified_at

9. API Plan

Method	Endpoint	Purpose
POST	/api/documents/upload	Upload a document
POST	/api/documents/{id}/process	Start processing
GET	/api/documents	List documents
GET	/api/documents/{id}	Get document and extracted result
PUT	/api/documents/{id}	Update corrected fields
POST	/api/documents/{id}/verify	Mark result as verified
DELETE	/api/documents/{id}	Delete document
GET	/api/health	Health check

10. Development Roadmap

Phase	Focus	Implementation
Phase 1	Foundation	Create repository, Python virtual environment, React app, FastAPI app and PostgreSQL connection.
Phase 2	OCR MVP	Implement upload, preprocessing and OCR. Return recognized text and bounding boxes.
Phase 3	Invoice Extraction	Implement invoice field extraction and structured JSON output.
Phase 4	Layout & Tables	Improve handling of complex layouts and line-item tables.
Phase 5	Document AI	Integrate and evaluate LayoutLMv3 or Donut for advanced document understanding.
Phase 6	Database	Persist documents, extraction results, line items and verification state.
Phase 7	React Dashboard	Build upload, processing, results, correction and document-history screens.
Phase 8	Visual Verification	Overlay OCR/extraction bounding boxes on the source document.
Phase 9	Testing	Unit tests, API tests, extraction tests and evaluation on a held-out dataset.
Phase 10	Docker	Create Dockerfiles and Docker Compose configuration.
Phase 11	Optimization	Improve latency, memory usage, batch processing and model loading.
Phase 12	Portfolio Release	Write README, architecture documentation, screenshots, metrics and resume bullets.

11. AI Evaluation Plan

The project should report measurable performance rather than only showing example outputs. OCR can be evaluated using character or word error rate. Field extraction can be evaluated using precision, recall and F1-score. Numeric fields should also be checked for exact or tolerance-based correctness. Processing latency and memory usage should be recorded for practical deployment.

- OCR: Character Error Rate (CER) and Word Error Rate (WER)
- Field extraction: Precision, Recall and F1-score
- Invoice totals: Exact/tolerance-based accuracy
- Document classification: Accuracy and F1-score
- System performance: Average processing time per document
- API: Response time and error rate

- Deployment: Memory consumption and container startup time

12. Security and Reliability Plan

- Restrict accepted file extensions and MIME types.
- Apply upload size limits.
- Generate safe server-side filenames instead of trusting uploaded names.
- Keep secrets in environment variables rather than source code.
- Validate extracted values before database insertion.
- Separate temporary uploads from persistent storage.
- Add API authentication if the system is later deployed publicly.
- Log processing failures without exposing sensitive document content unnecessarily.

13. Resume-Level Features

- Real-time document processing pipeline
- OCR with bounding-box visualization
- Transformer-based document understanding
- Structured invoice and table extraction
- Human-in-the-loop verification
- REST API with FastAPI
- PostgreSQL persistence
- React dashboard
- Dockerized multi-service deployment
- Automated testing and measurable model evaluation

14. Future Enhancements

- Support receipts, purchase orders, bills and bank statements.
- Add multilingual OCR.
- Add VLM/LLM-based question answering over documents.
- Add semantic search across processed documents.
- Add duplicate invoice detection.
- Add confidence-based human review queues.
- Add asynchronous background processing using Celery or a similar task queue.
- Add cloud deployment and object storage when the local version is stable.
- Add monitoring and model/version tracking.

15. Definition of Done for Version 1

- A user can upload an invoice from the React interface.
- The FastAPI backend validates and processes it.
- OCR extracts text and coordinates.

- The system extracts the main invoice fields.
- Results are returned as structured JSON.
- Results are stored in PostgreSQL.
- The dashboard shows extracted fields and source regions.
- The user can correct and verify extracted values.
- The application can be started using Docker Compose.
- The repository contains tests, documentation and reproducible setup instructions.

Recommended implementation principle: build the first working version with OpenCV + PaddleOCR + rule-based extraction before adding a large Transformer model. This keeps the project understandable, debuggable and measurable. Advanced Document AI models should be introduced after the baseline is working.